A04a · Judging Algorithms — Practice A2 · 9618 §19.1

Name: _____

Searching and sorting are examined **twice** — written out on Paper 3, typed on Paper 4 — so this homework has both halves. **Questions 1–6 are written, on this sheet. Questions 7–10 are code, in `A04a_Practice.java`** (listed at the bottom). One set of numbers across both.

1. Be the machine. This array is already in ascending order. [5]

3	7	12	18	21	25	30	34	41	47	52	58	63	70	88
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

A binary search runs on it, taking the mid-point as `Mid ← (Lower + Upper) DIV 2`, with `Lower` starting at 0 and `Upper` at 14.

(a) Complete the trace for **Target = 52**. One row per comparison; you may not need every row. [3]

Lower	Upper	Mid	value at Mid	< = or > Target

(b) How many comparisons did it take? [1]

(c) The search is run again with **Target = 60**, which is not in the array. State the condition that finally stops the loop. [1]

2. Fill the gaps. The array `Names[0:99]` is in alphabetical order. Complete this binary search. [6]

```
Lower ← 0
.............................................
Mid   ← 0
Exit  ← FALSE
OUTPUT "Enter the name to be found "
INPUT Target
REPEAT
   IF ..................................... THEN
      OUTPUT Target, " does not exist"
      Exit ← TRUE
   ENDIF
   Mid ← Lower + (Upper - Lower + 1) DIV 2
   IF Names[Mid] < Target THEN
      Lower ← ..............................
   ENDIF
   IF Names[Mid] > Target THEN
      ......................................
   ENDIF
   IF ..................................... THEN
      OUTPUT Target, " was found at location ", Mid
      Exit ← TRUE
   ENDIF
.............................................
```

3. Two letters and a sentence. Big O notation is used to classify the efficiency of algorithms. The Big O notation for the time complexity of a binary search is `O(log n)`. [3]

(a) State the Big O notation for the time complexity of a **linear search**. [1]

(b) Describe the meaning of `O(log n)` as it applies to a binary search algorithm. [2]

4. Two arrays, one sort. A club stores each member's unique ID number and their name in two arrays. The item at index 7 of one belongs to the item at index 7 of the other. [5]

```
DECLARE MemberID : ARRAY[0:499] OF INTEGER
DECLARE Names    : ARRAY[0:499] OF STRING
```

Write **pseudocode** for an **insertion sort** that puts `MemberID` into ascending order. The data in both arrays must move together, so that the data at any one index still applies to the same person. Use these identifiers; assume all are declared.

identifier	type	description
`Count`	INTEGER	main loop counter, to iterate through the whole array
`Temp1`	INTEGER	temporary holding location for the current member ID
`Temp2`	STRING	temporary holding location for the current member name
`Insert`	INTEGER	counter showing the current insertion position

5. The same algorithm, a different day. [4]

(a) Describe the factors, **other than changing the algorithm**, that affect the time the insertion sort in question 4 takes to run. [3]

(b) State the Big O notation for the **worst-case** time complexity of an insertion sort. [1]

6. Is it worth sorting first? A program holds **1 000** records in an array, in no particular order. Use these figures: [3]

> a linear search takes **500** comparisons on average · a binary search takes at most **10** · sorting the array first costs about **500 000** comparisons

(a) The program will search the array **five** times in total. Which approach does less work, and why? [1]

(b) How many searches must the program make before sorting first pays for itself? Show your working. [2]

On the computer — `A04a_Practice.java`

Fill in the four methods and run it — the judge is at the bottom of the file. Questions 7 and 8 are one real question from November 2024 Paper 41: a bubble sort that sorts *words*, using a comparison you have to write yourself.

7	**`CompareStrings`** — which of two words comes first alphabetically, one character at a time	[4]
8	**`BubbleStrings`** — bubble sort an array of words, using question 7 to compare	[3]
9	**`BinarySearch`** — iterative, returns the index or −1	[6]
10	**`MaxComparisons`** — *predict* the worst case of your own question 9, before the judge measures it	[2]

Written 26 + code 15 = **41 marks**.

Want to dig? Question 1 takes the mid-point as `(Lower + Upper) DIV 2`. Question 2 — a real paper — takes it as `Lower + (Upper - Lower + 1) DIV 2`. Those are *not* the same rule. Find the smallest array on which the two disagree about which index to look at first, and then answer the harder question: is either of them ever **wrong**, or do they just take different routes to the same answer? *(Test it in code. An array of two items is a good place to start looking.)*

Additional space. If you need more room for any answer, continue here and write the question number in the margin beside it.